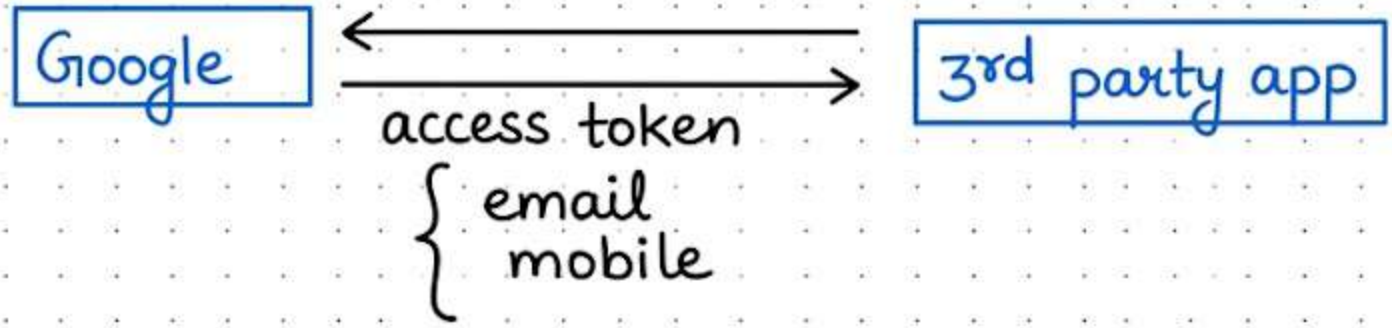
Lecture 11 Designing Search AutoComplete

↳ OAuth 2.0



Auto Complete Engine

Other name: Design top k most searched queries

Google

sw

- swing
- swim

O(1)

Questions:

Q: How many autocomplete suggestion is needed ?
↳ 6 auto complete suggestions

Q: How to decide the search result ?
↳ based on popularity (~~frequency~~)

Q: Search key length ?
↳ 50-100 (lets take 100)

Q: Which languages are supported ?
↳ English

Q: Should we need to sort the result ?
↳ yes

Q: Case Sensitive ?
↳ No

Q: Daily Active Users ?
↳ 5 million DAU

Requirements:

Search Auto complete Engine

1. Fast response time → O(1)
2. Sorted by popularity (trq)

(scalable and Availability over consistency) }

Back of the envelope calculations:

Assumptions:

- A person does 10 search result per day (average)
- DAU : 5million
- 30 bytes of data per query

1 character = 1 Byte }
30 char = 30 bytes }

Search queries:

Get : /search/q = sw
Get : /search/q = swa
Get : /search/q = swam

30 char → 30 queries on DB / search result.

QPS:

$$\Rightarrow \frac{5\text{million} * 10 * 30}{86{,}400}$$

$\Rightarrow 5 \times 2^{20} * 300 / 86{,}400$

$\Rightarrow 0.01736 * 2^{20}$

$\Rightarrow 17.36 * 2^{10}$

$\Rightarrow \sim 18{,}000$ QPS

Peak QPS : QPS * 2
18,000 * 2 = 36,000 peak QPS

Storage Unit:

Assume : for 5 years

$\Rightarrow 5 * 2^{20} * 10 * 30 * 365 * 5$

$\Rightarrow$ ~~31,80.~~ 27,37,500 $* 2^{20}$

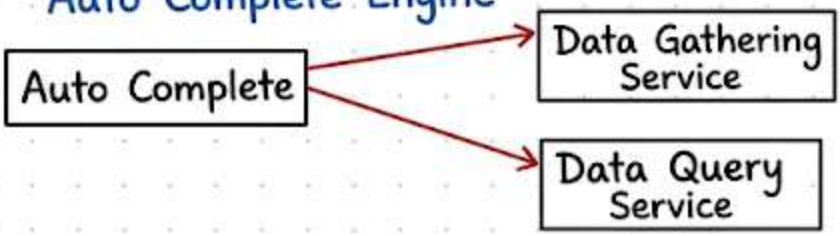
$\Rightarrow 2737.5 * 2^{30}$

$\Rightarrow 2.7375 * 2^{40}$

$\Rightarrow \sim 3$ TB

↳ High Level Overview:

Auto Complete Engine



- Data gathering service : → SQL
  Responsible for collecting information

- Data Query Service : → Queries
  Given a search query or prefix, return top 6 most searched result.

Data Gathering service :
SQL DB

Basic Frequency Table : (Initially Empty)

Query	Frequency
Swim	7
Swan	6
Swing	4
Swirl	3
Switch	2

Data Query Service :
Search : 'sw'
Swim
Swan
Swing
Swirl
Switch

How Query service works ?

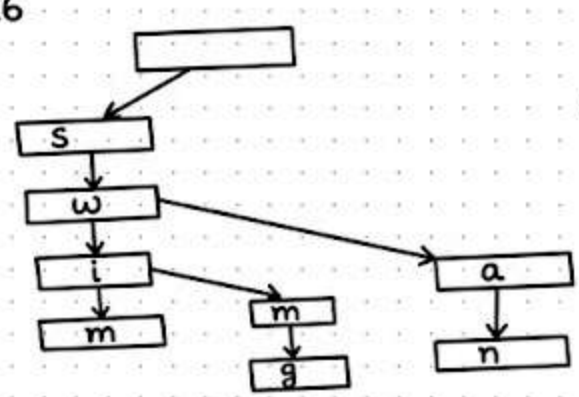
```
Select * from frequency where query like
                                ↳'@prefix' + 'sw'
order by frequency DESC
limit 6 ;
```

Work is Done :
Dataset is small → then this service is OK [O(n)]
Dataset is huge → Very very slow

Next Solution:

Tries Data Structure

English: 26



O(P)
↓
where p is length of the string

Modify normal Tries

problem with Normal Tries:

To implement autocomplete:

P: length of the prefix to be searched
n: Total no. of nodes in tries
c: No. of children of a given node

Top k most frequent queries:

1. Find the prefix. → O(P)
2. Traverse the subtree from this node to get all the valid children. → O(C)
3. Sort the result on the basis of freq → O(c log c)

Total time complexity → O(P) + O(C) + O(c log c)

worst case → traverse complete tries

How to make this fast?

1 Limit the max length of query
2 Cache top 6 results for each node

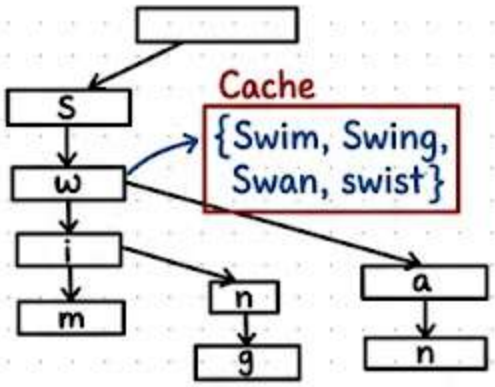
Limit the max length of string:

user rarely search for very long string
P is small (say 100 chars at max)
O(p) → O(1)

How to make this fast?

Limit max length of query

Cache top 6 results for each node



↳ Top k most frequent queries:

1. find the prefix → O(1)
2. Sort the result on the basis of freq → O(1)

Already sorted.

Total complexity → O(1) + O(1) = O(1)

Trade off → space increase to decrease time complexity

How to Cache?

Real time vs almost real time system

Real time search prediction → Almost Impossible

Throughput increase

Almost real time system

Why ?

↳ Top suggestions may not change much

↳ Not practical to update our tries at every request.

Real time vs Timely updates.

twitter → may use a more real time system than our engine

But not for us (Google search)

Analytics logs — Raw table about Search Queries
Not indexed

SQL Table : append only

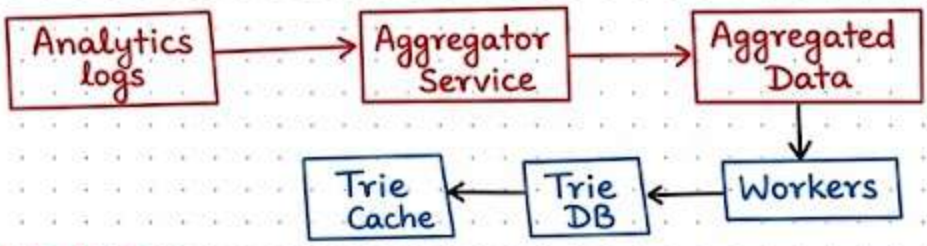
Query	time stamp
Swing	Aug 11 2025 22:48:03
Swan	Aug 11 2025 22:48:05



Aggregator Service :
Aggregate data to be processed further (format correction)
hon - real time system → I aggregate once a week
for more real time system → aggregate once a day.

Aggregated data : SQL Table

Query	time	freq
Swing	-	2
Swim	-	4



Workers:
Set of servers that perform async operations to build our Trie DS and store it in Trie Db

Trie Cache : - keep tries in memory for faster read.
- Hold the weekly updated information

Trie DB:

Persistent storage

Tries → key, value

(MongoDB / casandra)

Trie → serialize → key, value pair (Hashmap)

Key: Every prefix in the tries will be my key.

Value: Data for each trie node (top 6 results)

Key	Value
SW	Swim, Swan, Swing, swirl

First time service run:

1 lakh analytics log in one week

↓

10K aggregated data

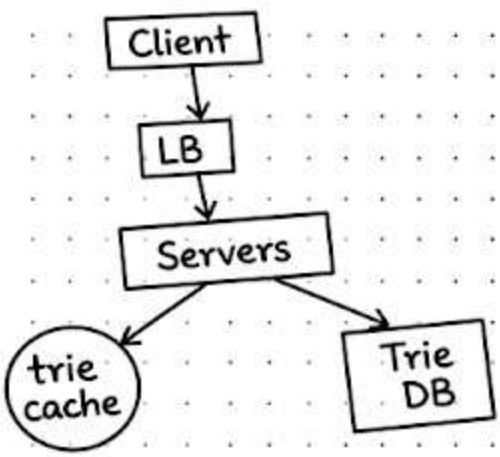
↓

Build tries out of it

↙ ↘

trie DB

trie cache



Flow:

1. A search query is sent to the LB.
2. The LB route this to our server.
3. Server tries to fetch data from cache DB if exist.
4. If not exist, replenish data from DB to cache (If cache is full or down).

Further Optimisations?

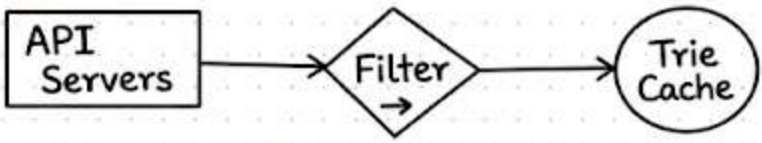
1. Client
   /search/query = sw
   /search/query = swim

   HTTP Req (not use)
   AJAX Req (use this)

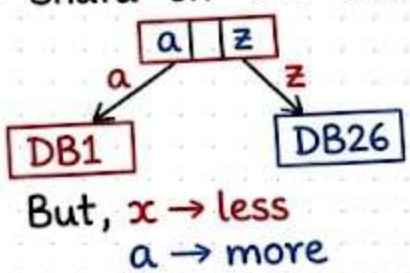
   Browser → sent ajax req, for auto complete. It will not refresh out web page at each letter.

2. To decrease the load ~~acca~~ on our server: we can also cache result at browser side (client side)

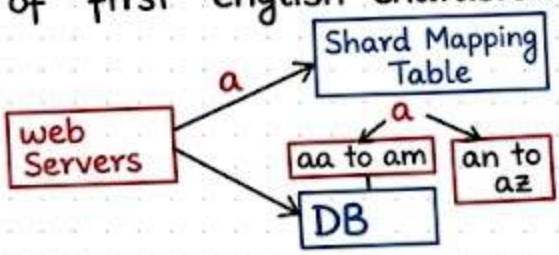
3. Offensive/abusive words can also come



4. Scaling the DB
   Sharding key ??
   - Shard on the basis of first english character.



But, x → less
a → more



5. Multiple language ??

Hinglish → hindi in english letters

Unicodes character → encoding standards for all characters of every language in the world.

6. Diff country → diff servers

we can have country specific top results

CDN (cache) for better response.